

HPC and AI Benchmarks on AICR

GPU, Interconnect, and Storage

Shaohao Chen

ORCD, MIT

AICR benchmarking working group

Outline

A bottom-up tour of the AICR cluster

- **Cluster & hardware overview** — B200, RTX 6000, IB, Vast
- **GPU compute** — gpu-fryer GEMM throughput
- **Interconnect** — NCCL collectives
- **End-to-end AI training** — Megatron-LM (with PyTorch)
- Storage — **multinode PyTorch I/O sweep** (dataloader, torch_save, DCP)
- **Key takeaways** and recommendations

AICR cluster — hardware at a glance

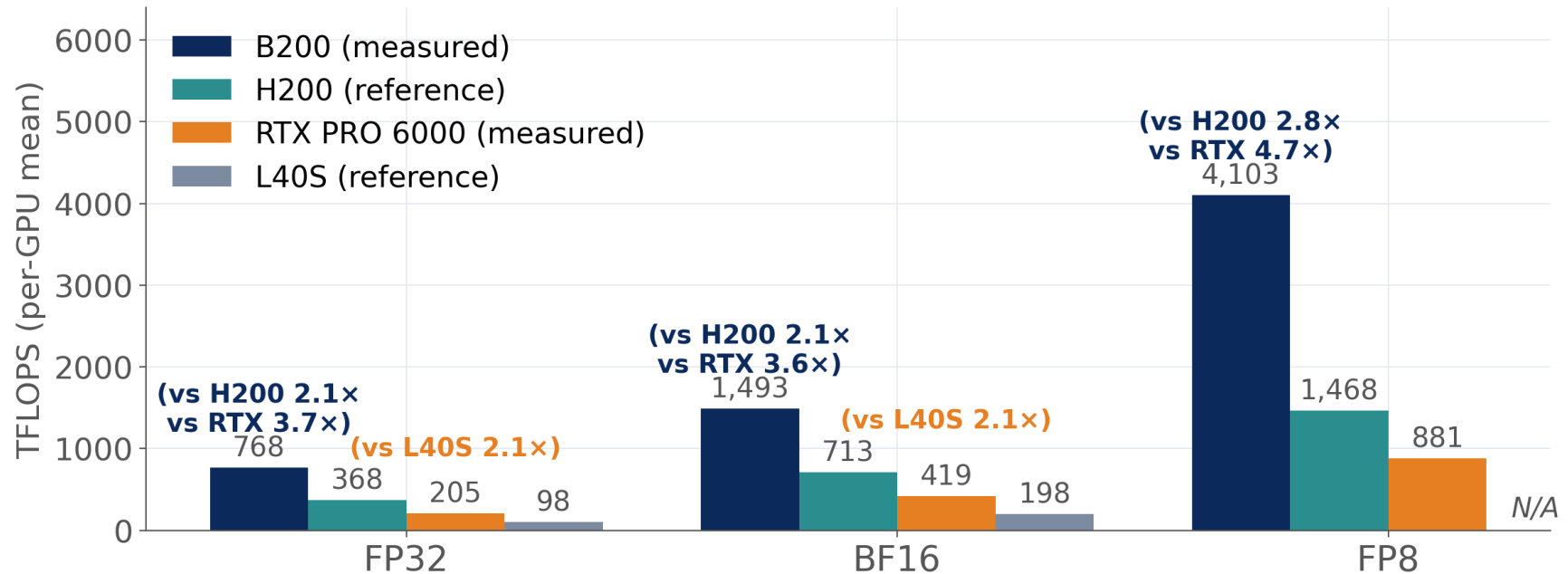
	B200 node	RTX PRO 6000 node
GPUs / node	8× NVIDIA B200	8× RTX PRO 6000 Blackwell SE
GPU memory	192 GB HBM3e	96 GB GDDR7
Intra-node interconnect	NVLink 5.0 + NVSwitch (~900 GB/s/GPU/dir)	PCIe Gen5 + Infinity Fabric only
NIC config	8× NDR 400 Gb/s (1 per GPU)	NDR IB (per-node)
CPU topology	2× AMD EPYC 9575F, NPS=4 (8 NUMA dies)	2× AMD EPYC 9575F, NPS=4 (8 NUMA dies)
Storage backend	Vast NFS over RDMA (Gen5 / Ceres 1350)	same

Fixed system issues

- **nvidia_peermem** loaded — enables GPUDirect RDMA over IB
- **ACS disabled, iommu=off** — restores PCIe P2P
- Applied to **both** B200 and RTX PRO 6000 nodes

GPU compute — gpu-fryer dense Tensor-Core GEMM

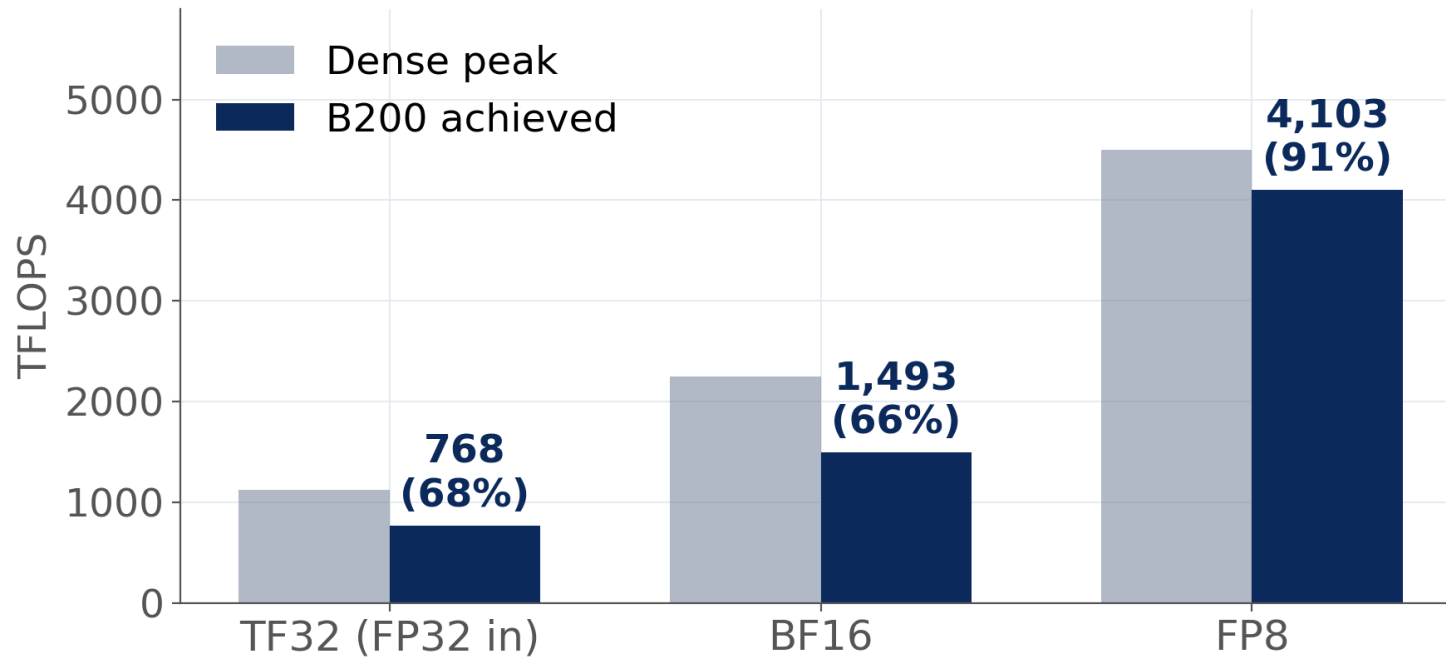
Per-GPU mean across 8 GPUs · gpu-fryer



- B200 $\approx$ **2.1x H200** (FP32/BF16), rising to **2.8x at FP8** — clearly optimized for low precision
- **BF16 for training. FP8 for inferencing.**
- B200 $\approx$ **3.6–4.7x RTX PRO 6000** (same arch, different die)
- All available nodes report "**All GPUs healthy**".

B200 — achieved TFLOPS vs dense peak

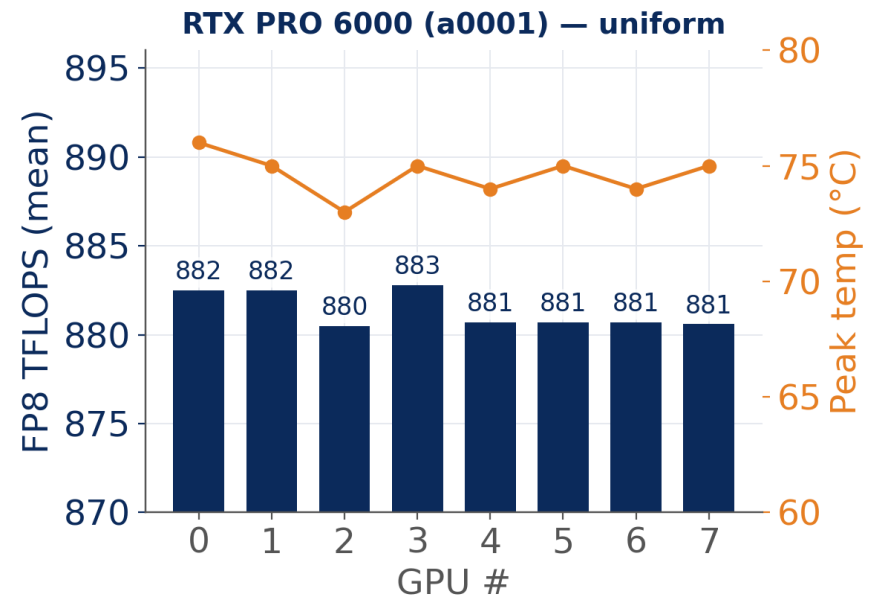
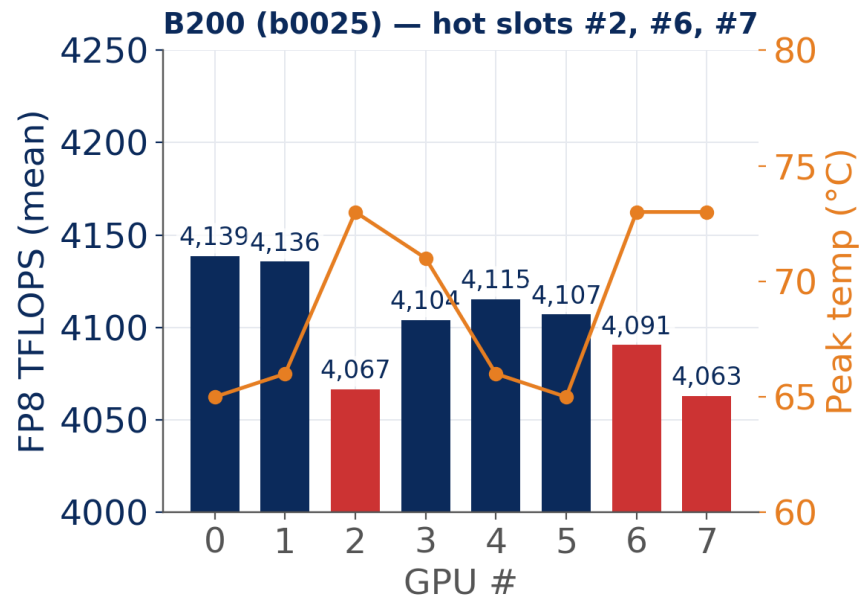
TF32 (FP32 input), BF16, FP8 · per-GPU mean



- FP8 is the standout — **91 % of dense peak**
- $\text{peak_FLOPS} = (\text{\#Tensor Cores}) \times (\text{ops per Tensor Core per cycle for that precision}) \times (\text{boost clock})$

Per-GPU uniformity and thermals

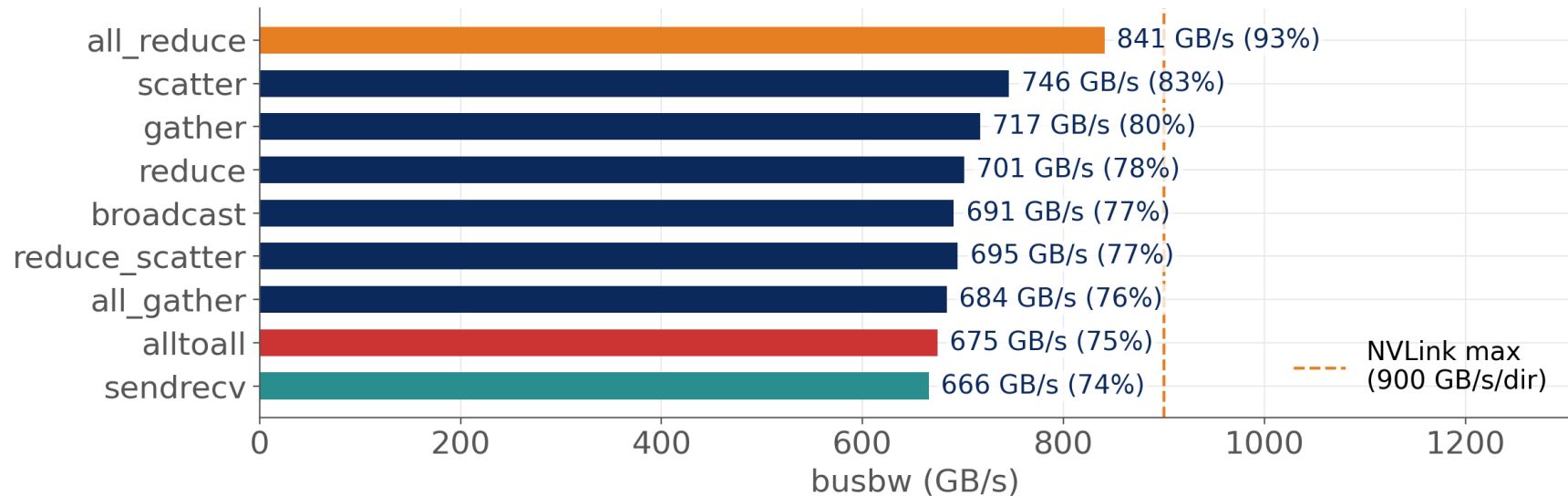
Healthy silicon, but B200 chassis shows three persistent hot slots



- TFLOPS are **uniform** across all GPUs on both B200 and RTX PRO 6000 nodes.
- Three B200 GPUs (#2, #6, #7) run **slightly warmer at 73 °C** — consistent with liquid-cooling topology, where coolant warms as it passes upstream GPUs before reaching downstream slots. All readings remain **within normal operating range**.

Interconnect — NCCL intra-node, B200 (NVLink 5 / NVSwitch)

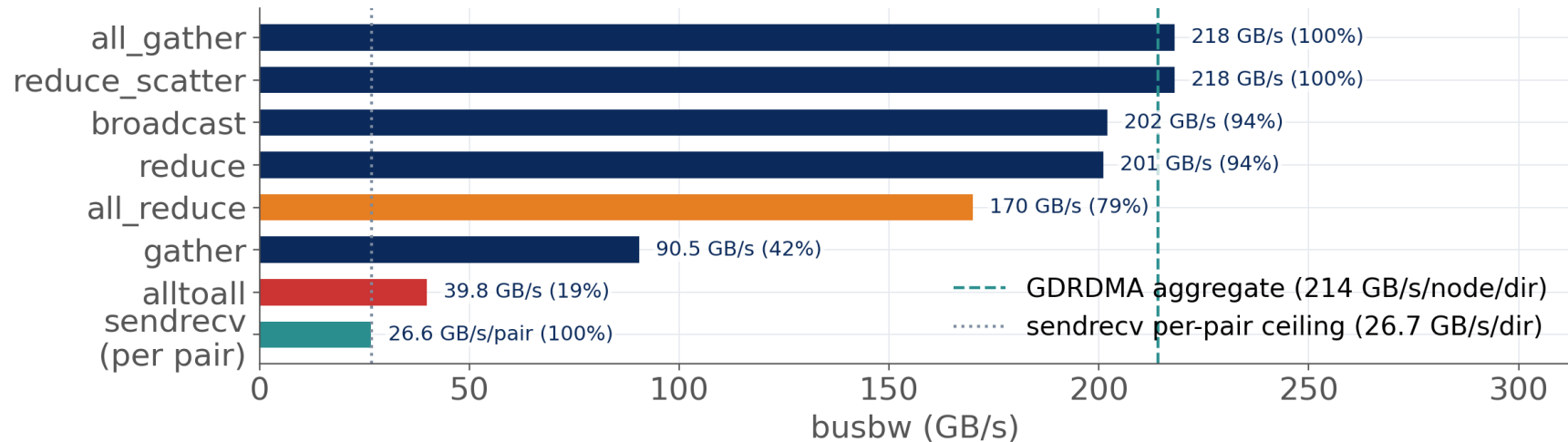
8 GPUs on b0027 · 16 GB message · NVLink max = 900 GB/s/dir



- All collectives achieve high bandwidth, enabled by the **NVLink 5 / NVSwitch** fabric.
- **AllReduce best at 93 %** (841 GB/s) — NCCL uses NVSwitch-optimal NVLS, not a ring; near-saturates the fabric.
- **SendRecv at 74 %** (666 GB/s) — NVSwitch P2P arbitration overhead; bidirectional traffic reduces peak unidirectional speed.
- **AllToAll at 75 %** (675 GB/s) — N^2 P2P algorithm overhead; lowest among symmetric collectives, but still healthy intra-node.

NCCL inter-node, B200 (2 nodes × 8 GPUs, NDR IB)

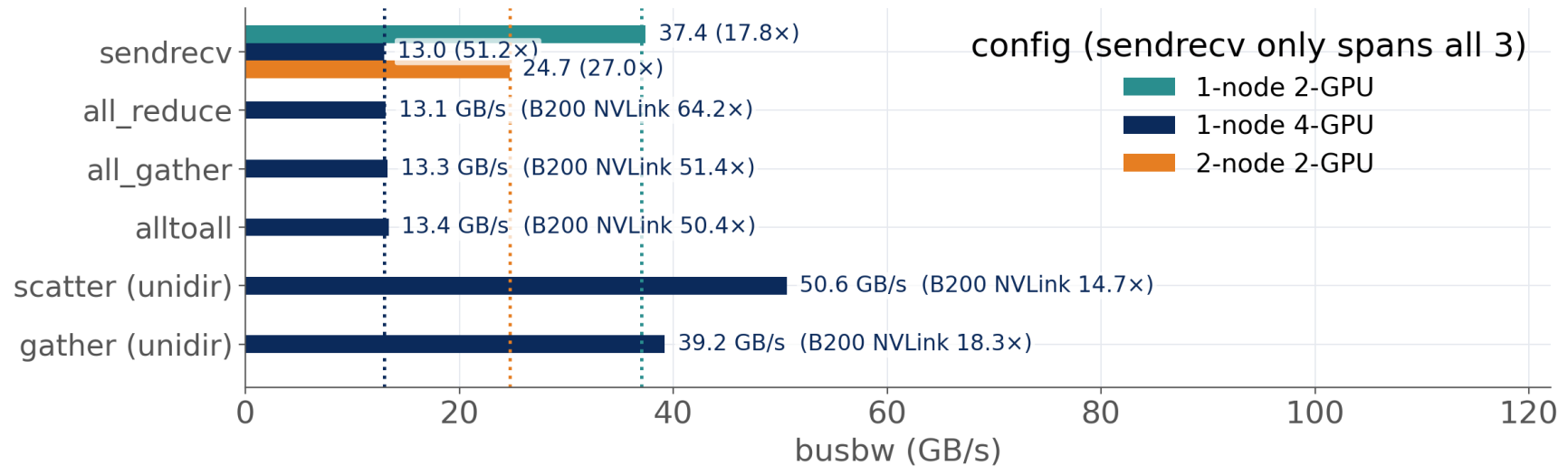
GDRDMA per-pair bidir ceiling = 26.7 GB/s/dir · aggregate ≈ 214 GB/s/node/dir



- **SendRecv at ~100 %** of the per-pair bidir ceiling (26.6 / 26.7 GB/s) — the GPU's PCIe DMA budget is shared between transmit and receive, halving the per-direction throughput vs the NDR unidir spec (50 GB/s). A **silicon-level constraint**; no NCCL tuning can lift it.
- **AllReduce** (170 GB/s) is good. Enabling SHARP could roughly double it to ≈ 340 GB/s. SHARP is in-network computing. The IB switch does the reduction.
- **AllToAll at 19 %** — NCCL ring-based N^2 P2P chunks are not pipelined across NICs; an **algorithm limitation**, not a hardware issue.

RTX PRO 6000 NCCL — no NVLink, PCIe is the bottleneck

NPS=4: every GPU pair traverses Infinity Fabric between NUMA dies · no GPU pair shares a PCIe switch · 16 GB message



- Three regimes, **three different bottlenecks** (dotted lines): **1-node 2-GPU** hits the GPU's PCIe DMA bidir budget (~37 GB/s); **1-node 4-GPU** saturates Infinity Fabric (~13 GB/s); **2-node 2-GPU** sendrecv hits the GDRDMA bidir DMA budget on the GPU (~24.7 GB/s). **1node 8-GPU** low speed at 0.04 GB/s — an issue to be fixed.
- The **2 → 4 GPU drop** happens because no GPU pair on this node shares a PCIe switch: every transfer must cross IF between NUMA dies, and once 4 pairs run concurrently the on-package fabric saturates.
- **vs B200 NVLink**: collectives are 15–64× slower on RTX PRO 6000; root-anchored unidir scatter/gather fare best because they avoid IF's bidir contention.
- Recommendation: **RTX PRO 6000 is best for single-GPU or 2-GPU workloads.**

Implications for parallel training (B200)

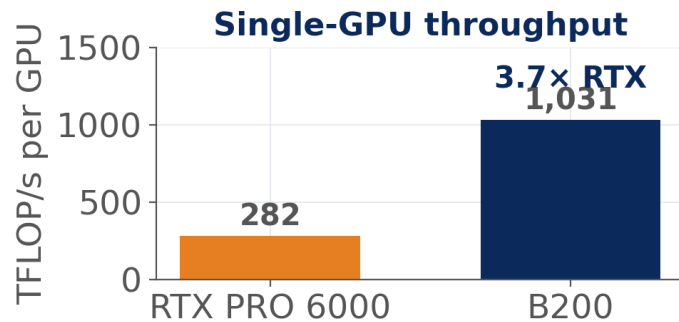
Mapping NCCL primitives to DL parallelism patterns

Parallelism	Primary NCCL op	1-node busbw	2-node busbw	Outlook
Data Parallel (DDP)	AllReduce	841 GB/s	170 GB/s	Inter-node-bound
Pipeline Parallel	SendRecv (P2P)	666 GB/s	26.6 GB/s	Inter-node at hard PCIe-DMA HW limit
Tensor Parallel	AR / AG / RS	684–841	170–218 GB/s	Keep within 1 node; cross-node ~4× slower
MoE (expert dispatch)	AllToAll	675 GB/s	39.8 GB/s	Inter-node severely bottlenecked (19% of agg.)

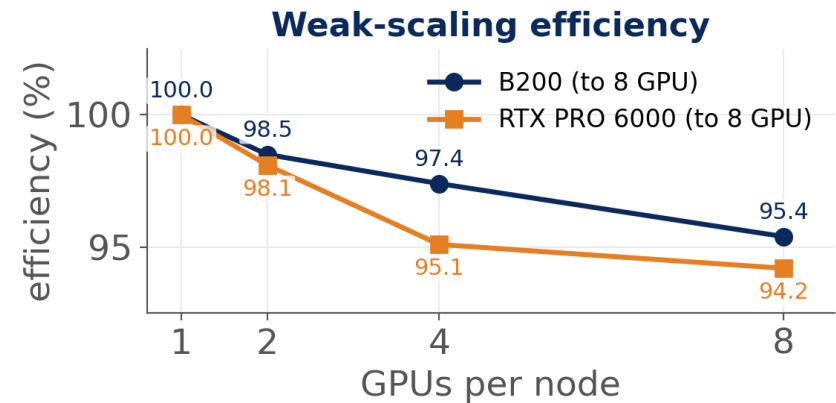
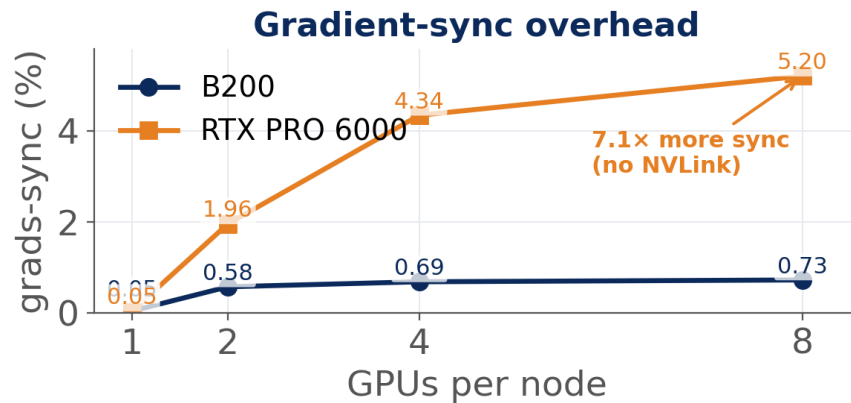
- All collectives perform well **intra-node**; **inter-node** performance varies by collective.
- **DDP scaling**: activate SHARP — gradient sync drops from 1032 ms to ≈ half.
- **PP across nodes**: 1 GB activation → ~37 ms over 26.6 GB/s sendrecv. Plan pipeline bubbles accordingly.
- **TP** > 8 ranks must cross nodes — drops from 841 to 218 GB/s; **prefer staying within a node**.
- **MoE at scale**: constrain experts to single node, or use NVSHMEM-based / custom AllToAll.

Megatron-LM end-to-end — GPT 1.32B, BF16, TP=PP=1 (Data Parallel only)

Single-GPU throughput + intra-node weak scaling — B200 vs RTX 6000 (per-GPU work held constant)

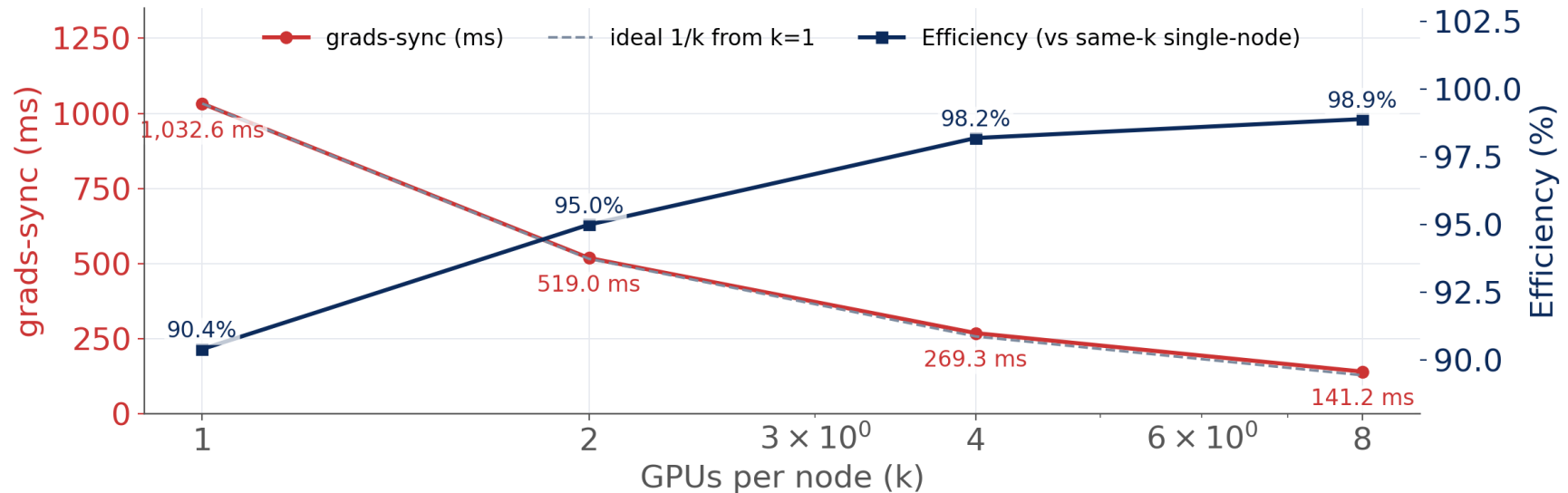


- **Data Parallel only**: each GPU holds a full model replica; gradients all-reduced every step.
- B200 delivers **3.7× per-GPU** throughput vs RTX 6000.
- B200: **95.4 % efficiency** at 8 GPUs — grads-sync stays < 1 %.
- RTX 6000: sync climbs to 5.2 % at 8 GPUs (**no NVLink**).



Megatron-LM — B200 multi-node scaling depends on GPUs/node

2-node runs · packing more GPUs/node shifts work onto NVLink



- **InfiniBand is the bottleneck** for gradient sync — each NIC carries a fixed share of the inter-node traffic, so grads-sync scales as $\approx 1/k$ with $k = \text{GPUs/node}$ (8 IB cards, 1 per GPU, run in parallel).
- **NVLink does most of the work** at 8 GPUs/node: it reduces gradients down to a $1/k$ slice intra-node, leaving only that small residual for IB to ship between nodes.
- Efficiency isolates the inter-node cost only — a healthy **98.9 %** at 8 GPUs/node.

Microbenchmark → real training

Why Megatron TFLOP/s/GPU is lower than gpu-fryer

GPU	gpu-fryer BF16	Megatron (1 GPU)	Megatron / gpu-fryer	MFU (vs dense peak)
B200	1,493	1,031.6	69 %	46 % of 2,250
RTX PRO 6000	419	279.5	67 %	(peak n/a)

Wall-clock step time includes many operations that consume time but are excluded from the FLOP count:

Cost in step time	FLOP-counted?
Forward/backward GEMMs (varied shapes, some small)	Yes — but lower Tensor Core utilization than microbenchmark tile
LayerNorm, softmax, dropout, activations, residuals	No — memory-bound
Adam optimizer step (BF16 grads → FP32 master weights)	No — memory-bound
Gradient all-reduce	No — communication
Kernel launch / Python / scheduling overhead	No

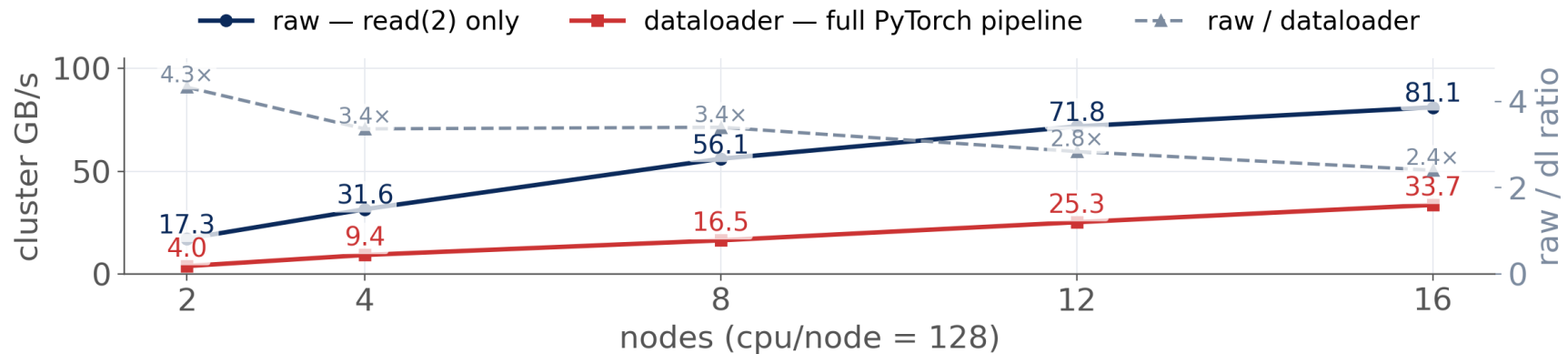
- **46 % MFU** on B200 sits at the high end of the published 30–50 % transformer pretraining range — the microbenchmark-vs-real-training gap is **normal, not a bug**.

Storage — multinode READ benchmark

PyTorch DataLoader on Vast NFS · ImageNet JPEG (21 GB, 186K files) · aggregate cluster GB/s (GPU1)

Two modes benchmark the read path using PyTorch on a Vast NFS over RDMA filesystem:

- **raw** — bare OS read(2) syscalls; upper bound of the storage path with no processing overhead
- **dataloader** — full PyTorch pipeline: NFS read → PIL JPEG decode → Resize(256) → CenterCrop(224) → ToTensor

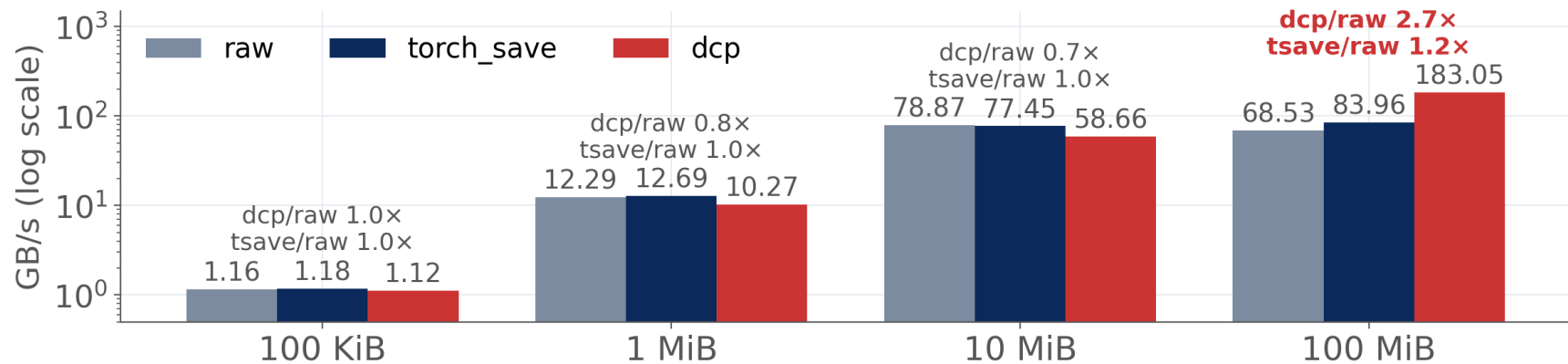


- raw / dataloader ratio falls from **4.3x** to **2.0x** as the cluster grows: JPEG decode capacity scales with added CPUs while raw read bandwidth saturates.
- 81 GB/s raw read = **18 % of the 462 GB/s Vast Max Read (client-bound)**; reading many small JPEG files is far slower than large sequential block reads.

Storage — multinode WRITE benchmark

raw / torch_save / dcp · GB/s at 16 nodes × 128 cpu/node · 256 files per rank

- **raw** — bare OS write(2) syscalls; upper bound of the storage write path
- **torch_save** — torch.save() pickles a float32 tensor; serial, **single-GPU checkpointing** (one process writes one file).
- **dcp** — torch.distributed.checkpoint: **multi-GPU checkpointing**, every rank writes its own shard in parallel.



- **raw** ≈ **torch_save** at every file size (within ±5 %), matching the 87.5 GB/s Sustained Write at large files.
- Max raw write **78.9 GB/s** (10 MiB) = **90 % of the 87.5 GB/s Vast Sustained Write** (48 % of the 165 GB/s burst Max) — near the array's steady-state write ceiling; at 12 nodes it peaks at 90.5 GB/s = **103 % of Sustained**.
- dcp lags raw by 16–26 % at 1–10 MiB (sharded-metadata overhead), then **wins 2.7× at 100 MiB** due to coordination of processes.
- Small files (100 KiB) are **metadata/IOPS-bound** — all three modes top out near the same ~1.16 GB/s ceiling.

Key takeaways

- **GPU** — B200 is healthy and $\sim 2.1\times$ H200 (FP32/BF16), $2.8\times$ at FP8; reaches 91 % of dense FP8 peak.
- **Interconnect intra-node** — B200 NVLink/NVSwitch is excellent (74–93 % of max, AllReduce at 93 %).
- **Interconnect inter-node** — B200 NDR-IB saturates the GDRDMA bidir limit (~ 214 GB/s/node/dir aggregate, 26.7 GB/s/dir per pair).
- **RTX PRO 6000** — no NVLink; ~ 37 GB/s at 2 GPUs, Infinity Fabric saturates at ~ 13 GB/s for 4 GPUs; best used at 1–2 GPU.
- **Megatron** — B200 = $3.7\times$ per-GPU vs RTX; intra-node weak scaling (per-GPU work held constant) holds a near-ideal 95.6 % at 8 GPUs, with a strong 46 % MFU — excellent.
- **Megatron multi-node** — efficiency rises 90 $\rightarrow$ 99 % as GPUs/node goes 1 $\rightarrow$ 8 (NVLink absorbs intra-node leg).
- **Storage reads** — client-bound at 81 GB/s (18 % of Vast Max Read); JPEG small-file reads lag large sequential blocks. DataLoader stays decode-bound.
- **Storage writes** — raw / torch_save match the 87.5 GB/s Sustained Write. DCP wins big at large files / many nodes (183 GB/s).

Recommendations & next steps

- **Activate SHARP** for B200 inter-node AllReduce — highest-leverage single change for DDP.
- **Monitor B200 hot-slot trio** (#2 / #6 / #7); investigate chassis airflow if gap widens.
- **Keep Tensor Parallel within one B200 node**; constrain MoE expert routing to a single node.
- Treat RTX PRO 6000 nodes as **1–2 GPU workhorses**; do not target multi-GPU collective-heavy workloads.
- For checkpointing large tensors at scale: prefer **torch.distributed.checkpoint (DCP)** over torch.save.

Thank you — questions?

Shaohao Chen · ORCD MIT · shaohao@mit.edu